

20-MINUTEN DEEP DIVE: LLAMA.CPP

llama.cpp

Lokale LLM-Inferenz ohne Cloud-Zwang & Overhead

Wann llama.cpp? · Single-Stream, CPU/Misch-Deployment, Apple Silicon & Edge.

Die Macht kleiner Apps: · Braucht dein Workflow (k)einen GPU-Cluster?

GGUF & Quantisierung: · Welche Quantisierung wähle ich wann?

Live-Demo & Setup: · Von Zero zu Token-Streaming in unter 60 Sekunden.



Die Inferenz-Landschaft: Fünf Engines im Vergleich

Merkmal	Ollama	llama.cpp	K-Transformers	vLLM	SGLang
Architektur-Trenner	Dynamic / On-Demand	Dynamic / On-Demand	Hybrid (CPU/GPU)	Pre-allocated High-Throughput	Pre-allocated High-Throughput
Speicher-Verwaltung (VRAM)	Allokiert dynamisch; gibt bei Inaktivität frei	Minimaler Basis-VRAM; allokiert KV-Cache zur Runtime	Hybrid: MoE im RAM, aktive Pfade auf GPU	Reserviert vorab ~90% VRAM für PagedAttention	Reserviert vorab fast gesamten VRAM (RadixAttention)
Hardware-Fokus	Lokale Rechner, Apple Silicon, Single-GPU	CPU-only, Apple Silicon, VRAM/RAM-Offload	Extremes Offload (große MoEs auf Heim-RAM)	Reine Datacenter-/Cloud-GPUs, Multi-GPU	Reine Datacenter-/Cloud-GPUs, Multi-GPU
Batching / Durchsatz	Gering (Single-User / wenige Anfragen)	Limitiert (primär sequenziell / kleiner Batch)	Limitiert auf Inferenz großer Hybrid-Modelle	Exzellent (Continuous Batching)	Exzellent + extrem schnell bei mehrstufigen Workflows
Einsatz & Fazit	Nur schnelles Ausprobieren; ungeeignet für Prod	Ideal für Prototyping, CPU-Mischbetrieb & Edge	Brücke: Riesen-MoEs auf Consumer-Hardware	Standard für Serving & Produktiv-APIs	Standard für Agenten, Structured Output & Multi-Turn

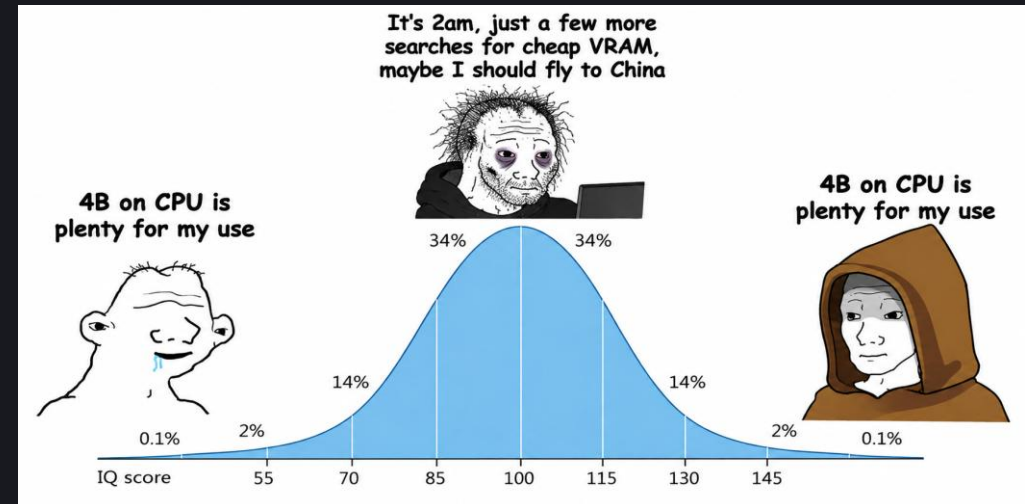
Positionierung: Wann llama.cpp?

WANN LLAMA.CPP? (DIE EINSATZFELDER)

1. Prototyping & schnelles Testen: Modelle in Sekunden ohne Docker, Registry oder Daemon-Setup starten und evaluieren.
2. Single-Stream Workflows: Interne Entwickler-Agenten, CLI-Tools, Hintergrund-Crawler, RAG-Pipelines.
3. CPU- & Misch-Deployment: Vorhandene Alt-Server oder Laptops nutzen, statt teuren GPU-Cloud-Speicher zu mieten.
4. Edge & Apple Silicon: Nutzt Unified Memory via Metal bis zum letzten Gigabyte ohne VRAM-Kopier-Overhead.
5. Mobile & Offline Apps: Pocket AI Lab (iOS) & LM Playground (Android) bringen GGUF direkt aufs Smartphone.

Die Macht kleiner Apps: Für 90% interner Workflows reichen 20–50 Tokens/s völlig aus (kein GPU-Cluster nötig!).

Hinweis zu Ollama: Für Hobbyisten nett – für planbare Latenz, Speicher-Audits und volle Parameterkontrolle ist llama.cpp der direkte Standard.



„4B on CPU is plenty for my use“

Die Bell Curve der lokalen Inferenz: Mehr Pragmatismus, weniger VRAM-Hype.

Euer KI-Einstieg: Seid ihr mit ChatGPT oder mit Claude gestartet?

TEAM CHATGPT / OPENAI

Der globale Einstiegspunkt für Millionen Entwickler

- GPT-3.5 (Nov. 2022): Der globale Aha-Moment für Text- & Chatgenerierung.
- GPT-4 (März 2023): Der Quantensprung: Echtes Reasoning & Code-Verständnis.
- GPT-4o (Mai 2024): Blitzschnell, multimodal – der weltweite Cloud-Standard.
- GPT-5 (Aug. 2025) & 5.2 (Dez. 2025): Der Sprung in autonomes Agenten-Reasoning.
- GPT-5.5 (Apr. 2026) & GPT-6 Astra (Sept. 2026): Test-Time Compute & Frontier-Spitze.
- **Frage an euch:** Wer von euch war bei GPT-3.5 / GPT-4 schon dabei?

TEAM CLAUDE / ANTHROPIC

Die Coding- & Nuancen-Referenz für professionelle Architekten

- Claude 3 Opus (März 2024): Erster Meilenstein, der GPT-4 im Coden spürbar deklassierte.
- Claude 3.5 Sonnet (Juni 2024): Der globale De-facto-Standard für Entwickler-Tools.
- Claude Opus 4 (Mai 2025) & 4.5 (Nov. 2025): Mathematisches Reasoning & Multi-File-Coding.
- Claude Opus 4.6 (Feb. 2026) & 5.5 (Sept. 2026): Die Benchmark-Spitze proprietärer Modelle.
- **Frage an euch:** Wer nutzt heute bevorzugt Claude für Architektur & Code?

DIE FUNDAMENTALE REFLEXION: erinnert ihr euch an das Gefühl, wie unerreichbar gut sich GPT-4 oder Claude 3 damals anfühlten? Schauen wir uns an, wo diese Modelle heute im Benchmark stehen – und wie viel dieser Intelligenz ihr heute vollkommen lokal und offline auf eurem Laptop betreiben könnt!

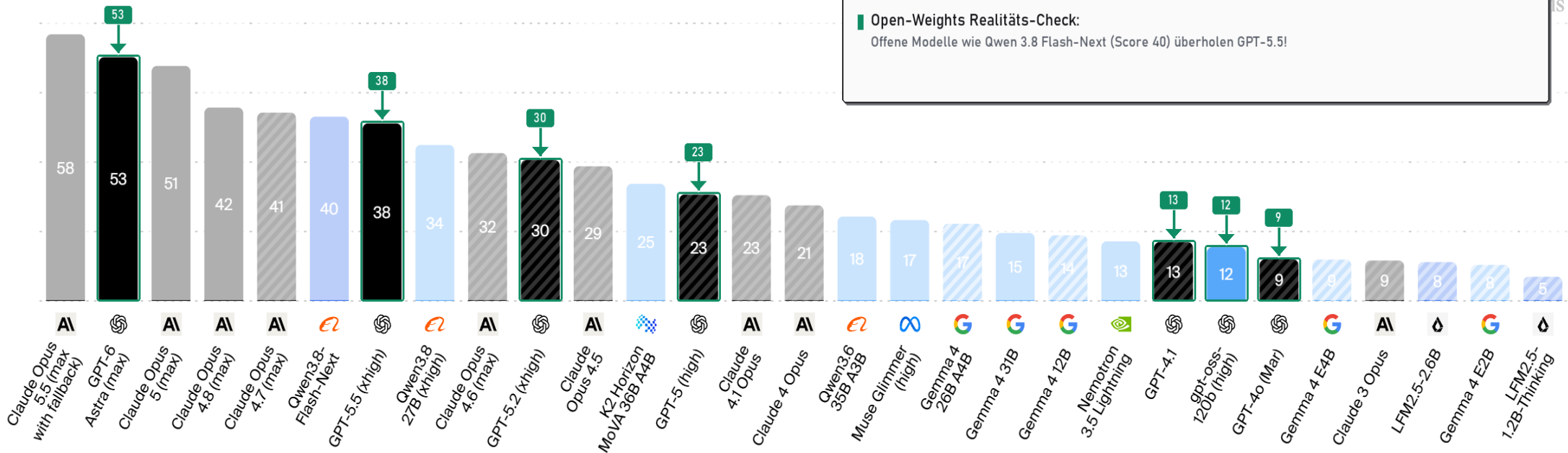
Benchmark 1: Wo steht die ChatGPT-Familie heute?

Artificial Analysis Intelligence Index by Open Weights / Proprietary

Artificial Analysis Intelligence Index v4.3.2 incorporates 10 evaluations: AA-Briefcase v1.1, GDPval-AA v2.1, AutomationBench-AA, Terminal-Bench 4.0, SciCode, Humanity's Last Exam, GDP.pdf, CritPt, AA-Omniscience, AA-LCR v1.1

▨ Estimate (independent evaluation forthcoming)

● Proprietary ● Open Weights (Commercial Use Restricted) ● Open Weights



OPENAI / CHATGPT IM BENCHMARK-VERGLEICH

- GPT-4o (Mar 2024) steht bei Score 9:
Genau dieses Niveau erreichen heutige 4B/9B Modelle komplett lokal.
- GPT-5.2 (Score 30) & GPT-5.5 (Score 38):
Enormer Sprung durch Test-Time Compute in der Cloud.
- Open-Weights Realitäts-Check:
Offene Modelle wie Qwen 3.8 Flash-Next (Score 40) überholen GPT-5.5!

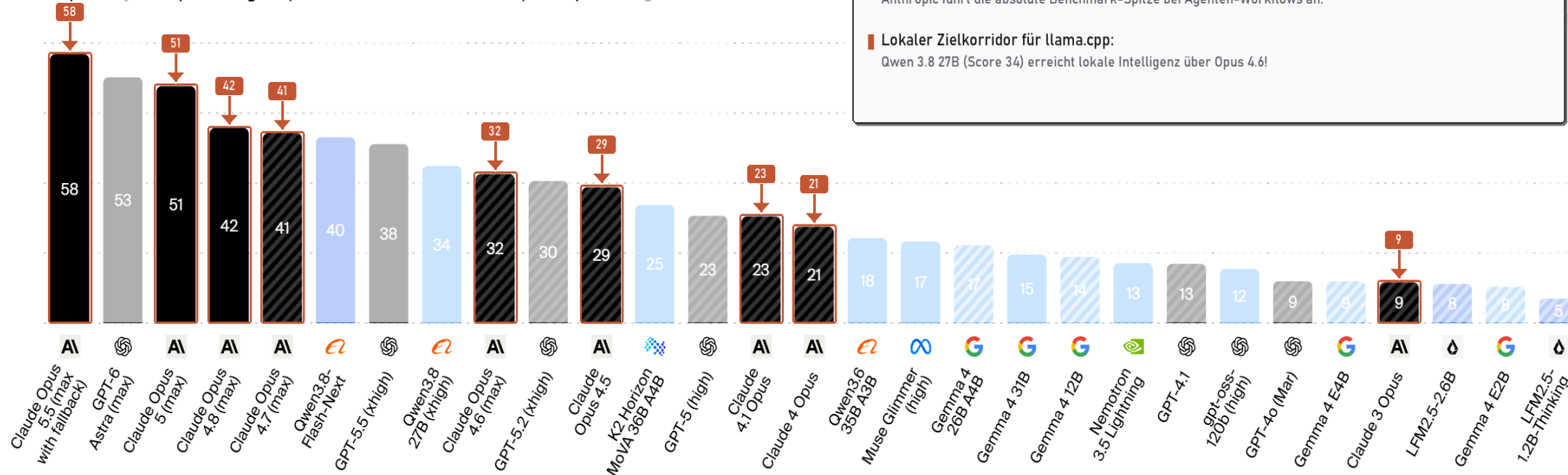
Benchmark 2: Wo steht die Claude-Familie?

Artificial Analysis Intelligence Index by Open Weights / Proprietary

Artificial Analysis Intelligence Index v4.3.2 incorporates 10 evaluations: AA-Briefcase v1.1, GDPval-AA v2.1, AutomationBench-AA, Terminal-Bench 4.0, SciCode, Humanity's Last Exam, GDP.pdf, CritPt, AA-Omniscience, AA-LCR v1.1

▨ Estimate (independent evaluation forthcoming)

● Proprietary ● Open Weights (Commercial Use Restricted) ● Open Weights



ANTHROPIC / CLAUDE IM BENCHMARK-VERGLEICH

- Claude 3 Opus (März 2024) steht bei Score 9:
Genau dieses Niveau erreichen heutige 4B bis 12B Open-Weights Modelle.
- Claude Opus 4.6 (Score 32) bis 5.5 (Score 58):
Anthropic führt die absolute Benchmark-Spitze bei Agenten-Workflows an.
- Lokaler Zielkorridor für llama.cpp:
Qwen 3.8 27B (Score 34) erreicht lokale Intelligenz über Opus 4.6!

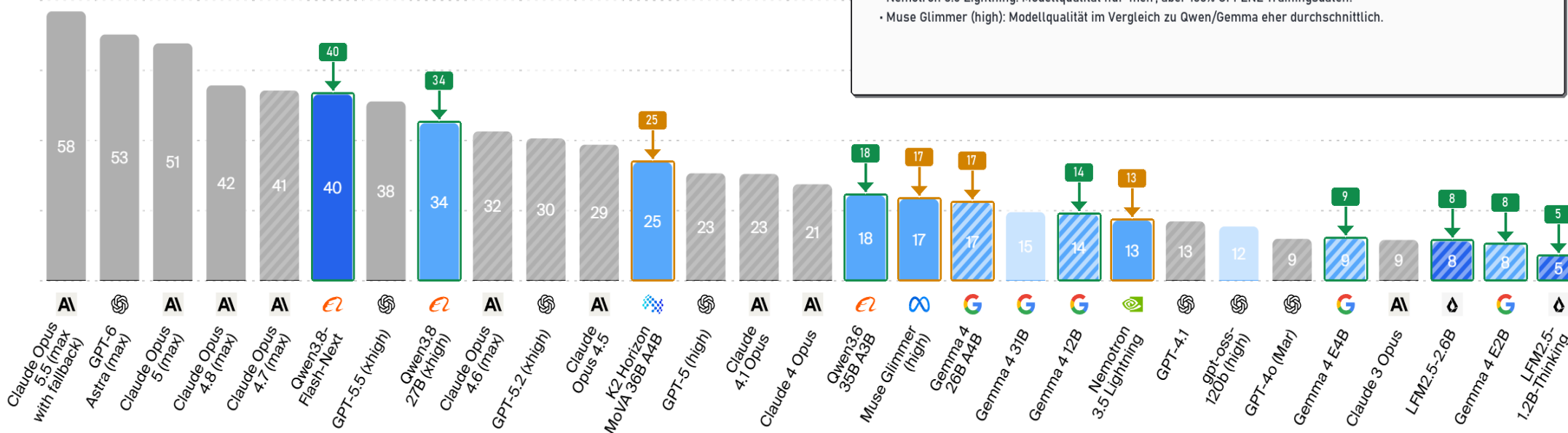
Benchmark 3: Open-Weights Realitäts-Check auf Augenhöhe

Artificial Analysis Intelligence Index by Open Weights / Proprietary

Artificial Analysis Intelligence Index v4.3.2 incorporates 10 evaluations: AA-Briefcase v1.1, GDPval-AA v2.1, AutomationBench-AA, Terminal-Bench 4.0, SciCode, Humanity's Last Exam, GDP.pdf, CritPt, AA-Omniscience, AA-LCR v1.1

▨ Estimate (independent evaluation forthcoming)

● Proprietary ● Open Weights (Commercial Use Restricted) ● Open Weights



OPEN-WEIGHTS REALITÄTS-CHECK: EMPFEHLUNGEN

GRÜN: KLARE DOWNLOAD-EMPFEHLUNGEN FÜR LLAMA.CPP

- Qwen 3.8 (Flash-Next & 27B): Übertrifft GPT-5.5 & Claude Opus 4.6 lokal!
- Gemma 4 (E2B, E4B, 12B): Native Audio- & Bildverarbeitung; E4B schlägt GPT-4o.
- LFM 2.5 (1.2B & 2.6B): Liquid AI – extrem schnell & sparsam auf CPU/Edge.

GELB: NUANCEN & VORBEHALTE BEACHTEN

- K2 Horizon MoVA (Score 25): Hohes Potenzial, aber Support in llama.cpp noch experimentell.
- Nemotron 3.5 Lightning: Modellqualität nur 'meh', aber 100% OFFENE Trainingsdaten!
- Muse Glimmer (high): Modellqualität im Vergleich zu Qwen/Gemma eher durchschnittlich.

Cheat Sheet: Die besten Open-Weights Modelle für llama.cpp

1. HIGH-END REASONING & CODING

Modelle & Benchmark: Qwen 3.8 (27B & Flash-Next) | Score 40 (schlägt GPT-5.5 & Claude Opus 4.6!)

Modellgewichte: Q4_K_M ~16.5 GB | Q8_0 ~29.5 GB

KV-Cache (8k Kontext): FP16 ~2.5 GB → Q8_0 (~1.2 GB via -ctk/-ctv q8_0)

Speculative Decoding: +0.8 GB für Drafter / MTP (liefert 2-3x Speedup)

VRAM-Gesamtbedarf: Q4: ~18.5 GB (ideal für 24 GB GPU / 32 GB RAM) | Q8: ~33 GB

2. MULTIMODALE ALLROUNDER (AUDIO + VISION)

Modelle & Benchmark: Google Gemma 4 (E4B & 12B) | Score 9 (Audio + Vision nativ!)

Modellgewichte: 4B: Q4 ~2.8 GB / Q8 ~4.5 GB | 12B: Q4 ~7.5 GB / Q8 ~13.5 GB

mmpoj (Vision/Audio): BF16 ~1.8 GB → FP8 nur ~0.9 GB (Geheimtipp AesSedai!)

KV-Cache (8k Kontext): FP16 ~0.8 GB → Q8_0 ~0.4 GB

VRAM-Gesamtbedarf: 4B in Q4: ~4.1 GB (Budget-PCs/6GB GPU) | 12B in Q4: ~8.8 GB (12GB GPU)

3. ULTRA-EFFIZIENZ & EDGE (CPU / MOBILE)

Modelle & Benchmark: Liquid AI LFM 2.5 (1.2B-Thinking & 2.6B) | Rasante CPU-Inferenz

Modellgewichte: 1.2B: Q4 ~0.8 GB / Q8 ~1.4 GB | 2.6B: Q4 ~1.7 GB / Q8 ~2.9 GB

KV-Cache & Drafter: LNN-Hybrid: Minimaler Cache-Bedarf (<0.3 GB), natives CoT-Thinking

VRAM / RAM-Bedarf: 1.2B: ~1.1 GB Total | 2.6B: ~2.1 GB Total (in Q4)

Hardware-Einsatz: Läuft extrem schnell auf CPU, Raspberry Pi & Smartphones (Pocket AI / LM)

4. 100% OFFENE TRAININGSDATEN & COMPLIANCE

Modell & Benchmark: Nvidia Nemotron 3.5 Lightning (8B) | Score 13 | 100% offene Daten

Modellgewichte (8B): Q4_K_M ~5.1 GB | Q8_0 ~9.2 GB

KV-Cache (8k Kontext): FP16 ~1.0 GB → Q8_0 ~0.5 GB (halbiert via -ctk/-ctv q8_0)

Drafter / MTP Head: +0.6 GB Drafter für schnelles Token-Streaming

VRAM-Gesamtbedarf: Q4: ~6.2 GB (ideal für 8 GB Laptop-GPUs) | Q8: ~10.3 GB (12 GB VRAM)

SOFTWARE-VORBEHALT: K2 Horizon MoVA 36B erreicht mit Score 25 hohes Potenzial, die native llama.cpp Unterstützung ist jedoch aktuell noch experimentell.

Das GGUF-Ökosystem: Warum es die Open-Source-Welt regiert

DIE 4 SUPERKRÄFTE VON GGUF (GPT-GENERATED UNIFIED FORMAT):

- 1. Single-File Container:** Modellgewichte, Tokenizer-Vokabular, Chat-Template & Metadaten in einer einzigen Datei. Nie wieder fehlende config.json!
- 2. mmap-Schnelligkeit:** Zero-Copy Memory Mapping. Modell startet in Sekundenschnelle direkt aus dem Dateisystem, ohne langwierige Deserialisierung.
- 3. Flexibles Layer-Offloading:** Granulare Verteilung: Z. B. 24 Layer auf die GPU schieben und die restlichen 8 Layer nahtlos auf der CPU rechnen lassen.
- 4. Hardware-agnostisch:** Läuft überall: x86 AVX2/AVX512, ARM64 (Apple Silicon Metal), NVIDIA CUDA, AMD ROCm, Intel SYCL & Vulkan.

Wo laden? Hugging Face! Die weltweite Plattform für Open Weights. Wer lädt neue Modelle am schnellsten hoch? Das schauen wir uns gleich an.



Quantisierungs-Mastery: Mythen, Fakten & Die Goldene Regel

16-Bit → 8-Bit (Q8_0): Immer machen!

Perplexitätsverlust < 0.05 PPL (menschlich ununterscheidbar).

→ Halbiert den VRAM/RAM-Bedarf und verdoppelt Bandbreiten-Effizienz! F16 ist reine Verschwendung.

4-Bit (Q4_K_M): Der Sweet Spot!

Reduktion auf ~25% der FP16-Originalgröße.

→ Moderne Tensor Cores (Nvidia Ampere/Ada/Blackwell, Apple AMX) bieten optimierte Matrix-Pfade für 4-Bit.

KV-Cache Quantisierung (Q8_0 / Q4_0): Mehr Kontext bei halbem VRAM!

Nicht nur Modellgewichte, auch der Kontext lässt sich quantisieren! ``-ctk q8_0 -ctv q8_0`` halbiert den Cache-VRAM bei langen Kontexten (z. B. 32k+) ohne spürbaren Qualitätsverlust.

5-Bit & 6-Bit: Rechnerisch ineffizient (Notlösung)

Hardware rechnet in 2er-Potenzen (4, 8, 16). 5- und 6-Bit müssen zur Laufzeit bitgepackt entpackt werden. Nur sinnvoll, wenn Q8 knapp nicht in den VRAM passt.

Unter 4-Bit (Q2, Q3): Finger weg in Production!

Dramatischer Qualitätsabsturz ('Perplexity Cliff'): Halluzinationen, Syntaxfehler im Code und logische Repetitionen nehmen massiv zu.

Faustformel:

Größeres Modell in 4-Bit schlägt kleineres Modell in 8-Bit!



GGUF-Quellen auf Hugging Face: Unsloth & Underdog AesSedai

DER PLATZHIRSCH: UNSLOTH

Repo: huggingface.co/unsloth

Das 'Release-Radar': Sobald Meta, Mistral oder Qwen neue Gewichte dropen, hat Unsloth oft nach 30 Minuten saubere GGUFs online.

Spezialität: Dynamic Quants (z. B. Q4_K_M mit partiellen FP16/Q8 Layern an kritischen Stellen) für minimalen Perplexitätsverlust.

Tipp: Ideal als erste Anlaufstelle, um zu prüfen, welche neuen Modelle gerade die Open-Source-Szene aufmischen.

DER GENIALE UNDERDOG: AESSEDAI

Repo: huggingface.co/AesSedai

Underdog-Story: Ähnlich herausragende Qualität wie Unsloth, wird aber oft übersehen, weil die Community nicht so gigantisch ist.

Der FP8-VRAM-Hack: **Normalerweise werden Vision Projector Files (mmproj-*.gguf) nur in BF16/FP16 bereitgestellt (1–3 GB extra VRAM). AesSedai liefert eine FP8-Version der mmproj-Files mit!**

Mix & Match: Die schlanken FP8-Vision-Dateien von AesSedai sind meist 100% kompatibel mit den Unsloth-Sprachmodellen!

Modell-Check bei Unsloth starten → Bei Vision-Modellen zu AesSedai wechseln und das FP8-mmproj laden → Wertvollen VRAM sparen und flüssiger arbeiten!

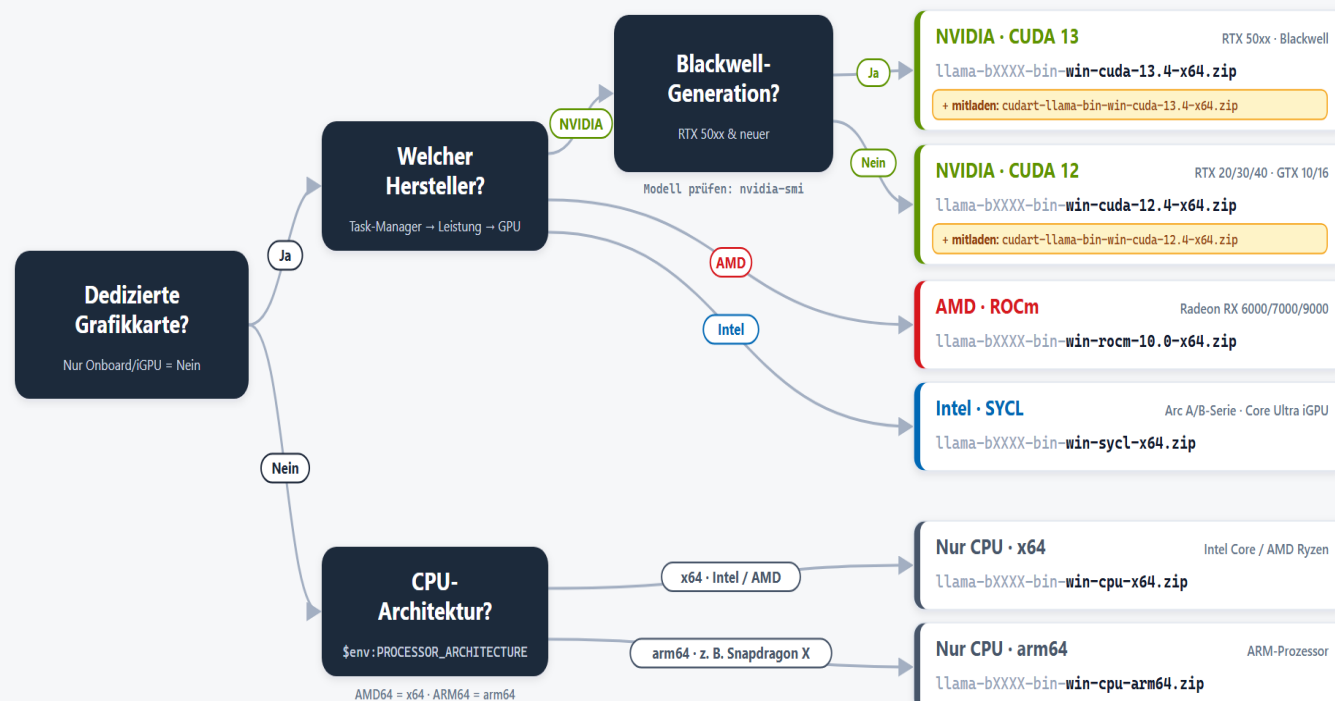
GitHub-Releases entschlüsselt: Windows & Linux

llama.cpp Download-Kompass

Quelle: github.com/ggml-org/llama.cpp/releases → neuestes Release → „Assets“ aufklappen

Alle Dateien heißen `llama-bXXXX-bin-...` (XXXX = aktuelle Build-Nr., z. B. b11377) **Gelb** = zweites Archiv (CUDA-Runtime) laden & in DENSELBE Ordner entpacken

WINDOWS



1 Herunterladen

Datei(en) aus dem Baum oben laden

2 Entpacken – alles in EINEN Ordner

zip entpacken (Rechtsklick → Alle extrahieren)

3 Modell (.gguf) dazu → Terminal → starten

`./llama-server.exe -m modell.gguf → localhost:8080`

Bewusst weggelassen: Vulkan (Universal-Fallback, z. B. ältere AMD-Karten / iGPU), OpenVINO, OpenCL-Adreno, NVIDIA auf arm64 (win-cuda-13.4-arm64), ROCm/SYCL: aktuellen GPU-Treiber installieren.

DIE GOLDENEN REGELN FÜR WINDOWS & LINUX:

NVIDIA (RTX 50xx / Blackwell): CUDA 13.4 Release wählen (z. B. win-cuda-13.4-x64.zip).

NVIDIA (RTX 40xx / 30xx / 20xx): CUDA 12.8 Release wählen (stabilste Performance).

DER CUDART-FEHLER:

Nvidia-Builds erfordern ZWEI Archive: Das Binary-Archiv UND cudart-...zip! Beide in denselben Ordner entpacken!

Linux / Ubuntu: Exakt dieselbe Logik: ubuntu-cuda-12.8-x64.tar.gz + cudart.

AMD Radeon (Linux): ROCm 10.0 Archive nutzen (kein CUDA nötig).

Ohne dedizierte GPU: Immer das CPU-Build mit AVX2/AVX512 Support wählen!

Apple Silicon, Mobile & Edge: Mac, iOS & Android

llama.cpp Download-Kompass

Quelle: github.com/ggml-org/llama.cpp/releases → neuestes Release → „Assets“ aufklappen

Desktop = Terminal-Weg mit Release-Dateien · Smartphone = fertige Wrapper-App nehmen (laufen unter der Haube auf llama.cpp / GGUF)

APPLE & MOBILE

macOS

Terminal-Weg

Welcher Chip?

→ Über diesen Mac



„Kann nicht geöffnet werden“? Gatekeeper-Sperre im Ordner lösen:
`xattr -dr com.apple.quarantine .`

1 Bequemer: Homebrew
`brew install llama.cpp`

2 Starten
`./llama-server -m modell.gguf`

Gibt es auch: macos-arm64 „KleidAI“ – aktuell deaktiviert, ignorieren.

Warum auf dem Smartphone eine Wrapper-App?

iOS/Android haben kein offenes Terminal & sandboxen Apps · App übernimmt Download, Speicher & Thermik · gleiche GGUF-Modelle wie am PC

iOS / iPadOS

App-Weg

EMPFEHLUNG · Wrapper-App

Pocket AI Lab

App Store

im App Store suchen

- Kein Terminal, keine Binaries
- App lädt & verwaltet Modelle
- Läuft komplett offline
- Realistisch: 1–4B in Q4
- Geräte mit 8 GB+ RAM ideal

NUR FÜR ENTWICKLER / BASTLER

Eigene App bauen: `...xcframework.zip`
→ in Xcode einbinden

Android

App-Weg

EMPFEHLUNG · Wrapper-App

LM Playground

lmplayground.app · kostenlos · Open Source

- Kein Terminal, keine Binaries
- App lädt & verwaltet Modelle
- Läuft komplett offline
- Realistisch: 1–4B in Q4
- Geräte mit 8 GB+ RAM ideal

NUR FÜR ENTWICKLER / BASTLER

Termux / adb: `android-arm64`
Snapdragon: `...arm64-snapdragon`
(Adreno GPU + Hexagon NPU)

DIE MOBILE & APPLE SUPERKRÄFTE:

macOS Apple Silicon (M1–M5):

llama-bin-macos-arm64.tar.gz
herunterladen. Metal ist standardmäßig
aktiviert!

Unified Memory Vorteil: CPU und GPU
teilen sich denselben 128 GB RAM-Pool – 70B
Modelle laufen ohne PCIe-Flaschenhals!

iOS App: Pocket AI Lab:

Kostenlose App für iPhone & iPad.
Ermöglicht das Laden eigener GGUF-Dateien
mit nativer Metal-Beschleunigung.

Android: LM Playground: Freie, Open-
Source Android App (lmplayground.app).
Nutzt Snapdragon NPU & Adreno GPU via
Vulkan.

Snapdragon X Elite / Laptops: Eigene
Binaries mit Hexagon NPU-Unterstützung
verfügbar!

In 3 Schritten starten & Die wichtigsten CLI-Flags

IN 3 SCHRITTEN ZUM STARTKLAREN LLM-SERVER:

1. ZIP entpacken: Release-Archiv herunterladen und in einen beliebigen Ordner entpacken (z. B. C:\llama oder ~/llama).
2. GGUF ablegen: Gewünschtes Modell von HuggingFace (z. B. via Unsloth oder AesSedai) direkt daneben speichern.
3. Server starten: `./llama-server -m qwen27b.gguf -c 8192 -ctk q8_0 -ctv q8_0 --port 8080` (Fertig! WebUI & API aktiv).

Flag / Parameter	Empfohlener Wert	Funktion & Performance-Impact
-m / --model	pfad/zum/modell.gguf	Pfad zur GGUF-Datei (Single-File Container mit Gewichten & Tokenizer).
-c / --ctx-size	8192 (oder 32768)	Kontextgröße in Tokens. llama.cpp allokiert den KV-Cache exakt passend dazu.
-ctk / -ctv	q8_0 (für Key & Value)	KV-Cache Quantisierung: Halbiert den VRAM-Bedarf langer Kontexte ohne Qualitätsverlust!
--mmproj	pfad/zum/mmproj.gguf	Multi-Modal Projector: Lädt Vision- & Audio-Encoder (Tipp: FP8 von AesSedai spart VRAM!).
-md / --draft-model	pfad/zum/drafter.gguf	Speculative Decoding / MTP: Nutzt kleines Drafter-Modell für 2-3x Speedup (v. a. Dense-Modelle).
-t / --threads	Anzahl physischer CPU-Kerne	CPU-Worker Threads. Regel: Niemals Hyperthreading-Kerne mitzählen (vermeidet Latenz-Spikes).
--port / --host	--port 8080 --host 0.0.0.0	Startet sofort eine OpenAI-kompatible REST-API + eingebaute WebUI im lokalen Netzwerk.

Zusatzmodelle: Vision (--mmproj) & Turbo mit Draft / MTP

VISION-MODELLE: DAS 2-DATEIEN-PRINZIP

Warum zwei Dateien? Sprachmodell (model.gguf) und Vision Projector (mmproj-*.gguf) sind getrennt.

Der Startbefehl:

```
./llama-server -m llama3-vision.gguf --mmproj mmproj-model-f16.gguf
```

Die Superkraft: Volle OCR-, Dokumenten- & Bildanalyse lokal auf dem Rechner. DSGVO-konform für sensible Belege!

Geheimtipp AesSedai: Nutzt das FP8-quantisierte mmproj von AesSedai auf HuggingFace! Spart VRAM und läuft genauso akkurat.

Native Alternative: Google Gemma 4 bringt Audio + Vision direkt integriert mit!

TURBO-INFERENZ: SPECULATIVE DECODING & MTP

Das Problem: Große Modelle sind Memory-Bandwidth-bound (jedes Token liest alle Gewichte neu ein).

Die Lösung (Draft Model): Ein winziges Draft-Modell (z. B. 0.5B) generiert 5 Token-Vorschläge blitzschnell.

Verifikation im Pulk: Das große Zielmodell (z. B. 27B) prüft alle 5 Tokens in EINEM EINZIGEN Vorwärtsdurchlauf!

Startflag in llama.cpp:

```
./llama-server -m qwen27b.gguf -md qwen0.5b.gguf
```

MTP (Multi-Token Prediction): Moderne Architekturen wie DeepSeek bringen eigene MTP-Heads mit – kein zweites Modell nötig!

Ergebnis: 1.5x bis 2.5x höhere Generierungsrate bei mathematisch exakt identischem Output!

LIVE-DEMO: llama-server in Action & API-Drop-In

DER LIVE-DEMO ABLAUF (3 MINUTEN):

1. Terminal-Start: `./llama-server -m qwen27b.gguf -c 8192 -ctk q8_0 -ctv q8_0`
2. Log-Inspektion: mmap Ladezeit (<1s!), VRAM-Allokation, Token-Timings in der Konsole.
3. WebUI im Browser: Aufruf von `http://localhost:8080` – sofort fertige Chat-Oberfläche.
4. Prompt-Caching: Zweite Frage: Cache spart Prompt-Rechenzeit komplett ein!
5. OpenAI Drop-In API: `http://localhost:8080/v1` – Jedes Tool funktioniert sofort:
 - **Coding in IDEs:** Continue.dev (VS Code & JetBrains), Cursor, Cline / Roo Code, Windsurf
 - **Chat & Knowledge:** Open-WebUI, LibreChat, AnythingLLM, Jan.ai
 - **SDKs & Notizen:** Python openai SDK (base_url), Obsidian Copilot, LangChain, LiteLLM

DIE PUBLIKUMS-WETTE:

Frage ans Publikum: Wie viele Tokens/s schafft das Modell auf dieser Maschine?

Typische Schätzungen: „5–10 T/s? Ist ja lokal...“

Die Realität:

Auf modernem Laptop/GPU: 35–70+ Tokens/s!
Schneller als das menschliche Auge lesen kann.

Prompt Processing vs. Gen: llama.cpp zeigt in den Logs genau an: Prompt eval time vs. Token generation time.

Takeaway für Skeptiker: Lokale Inferenz fühlt sich heute flotter an als viele überlastete Cloud-APIs zur Peak-Zeit!

Der Architektur-Kompass & Die 5 Goldenen Takeaways

DER ARCHITEKTUR-KOMPASS:

Single-Stream / CPU / Mac / Lokale Tools:

→ **llama.cpp** (Der unangefochtene König für schlanken Betrieb ohne GPU-Zwang.)

Riesen-MoE (DeepSeek) auf Heim-Hardware:

→ **K-Transformers** (Gewichte im DDR5-RAM, Rechenpfade auf 1-2 GPUs.)

High-Throughput Production-APIs:

→ **vLLM** (Continuous Batching & PagedAttention für hunderte parallele Nutzer.)

Agenten & Structured Output Multi-Turn:

→ **SGLang** (RadixAttention für extreme KV-Cache-Wiederverwendung.)

5 ERKENNTNISSE ZUM MITNEHMEN:

1. **llama.cpp** ist Production-Ready: Für Single-Stream, CPU, Mac und interne Tools die kostengünstigste Engine überhaupt.
2. **Ollama meiden**: Nutzt direkte Prebuilt-Binaries für volle Transparenz und planbare Latenz.
3. **Immer 8-Bit statt 16-Bit**: Q8_0 halbiert den RAM-Bedarf ohne messbaren Qualitätsverlust.
4. **Größeres Modell @ 4-Bit > Kleineres @ 8-Bit**: Lieber 27B Q4 als 14B Q8 – Intelligenz maximieren!
5. **Keine Angst vor C++**: Download-Kompass nutzen, ZIP entpacken, GGUF reinlegen, ./llama-server starten!

Handout, Ressourcen & GitHub Repository

GITHUB REPO, CHEAT SHEETS & GGUF-HUB

GitHub Repository: Vollständige Folien, Skripte, Benchmarks & Cheat Sheets als Open-Source Repo.

Download-Kompass Handout: Druckfertiges A4-Querformat PDF: [cheatsheets/handout.pdf](#) im Repository.

GGUF & Quant-Cheat Sheet: Kompakte Übersicht für VRAM-Rechnung, Sweet-Spots (Q4_K_M vs Q8) und CLI-Flags.

Unsloth AI (Hugging Face): [huggingface.co/unsloth](#) – Erste Anlaufstelle für brandneue GGUF-Drops & optimierte Quants.

AesSedai (Underdog-Tipp): [huggingface.co/AesSedai](#) – Exklusive FP8 mmproj-Dateien (Vision/Audio) & speichereffiziente Quants.

OFFIZIELLE ENGINES, CLIENTS & QUELLEN

llama.cpp Releases: [github.com/ggml-org/llama.cpp/releases](#) – Offizielle Binaries für Windows, macOS, Linux.

ggml Core Library: [github.com/ggml-org/ggml](#) – Die Low-Level C/C++ Tensor-Engine hinter llama.cpp.

Coding IDEs & Agenten: Continue.dev (VS Code / JetBrains), Cursor, Cline / Roo Code, Windsurf.

Lokale Web-UIs & Chat: Open-WebUI ([openwebui.com](#)), LibreChat ([librechat.ai](#)), AnythingLLM.

Mobile Apps (GGUF offline): Pocket AI Lab (iOS) · LM Playground (Android).

Benchmarks & Leaderboards: Artificial Analysis ([artificialanalysis.ai](#)) · LMSYS Arena ([lmarena.ai](#)).

VIELEN DANK FÜR EURE AUFMERKSAMKEIT! · ZEIT FÜR FRAGEN & DISKUSSION